Course Notes — SESSION 2
EVOLUTIONARY STRATEGIES

1. Introduction

In the previous session, we introduced the idea that credit may be assigned efficiently by viewing problem-solving as a search for merely satisfactory solutions. Yet recall our examples from the first session: the shipbuilder learning to build ships, the app developer learning to develop features, or the budget committee learning to invest. The real-world complexity of their problems suggested a vast search tree. Using heuristics to "prune" branches of this tree may still enable searching only a small part of it, implying that satisfactory solutions are not guaranteed. We may need additional ways to learn *where* in the tree to search effectively, not just how to search efficiently.

We will call processes of learning to search effectively for solutions to problems as ones of **adaptation**, and AI ideas for how to adapt as **evolutionary strategies**, in that they derive from Darwinism (more on this connection later). The overall strategy will be, rather than just assign credit to *sequentially*-interdependent actions (the focus of reinforcement learning), to assign credit across *spatially*-interdependent actions. Such an overall strategy, we will see, essentially fleshes out the logic of Simon's watchmaker parable. In the parable, the challenge is not really the sequence of the watch assembly tasks, but in the need for effective decomposition based on understanding how a large number of parts are spatially-configured to function together. Likewise, in this session, we will take the fundamental challenge of intelligence as essentially one of decomposing the parts of a problem according to their functions.

This focus on spatial interdependence will call for us to shift to a new way of thinking about what a program or plan is. From now on, we will treat programs as less like a sequential, step-by-step procedure, and instead more like diagrammatic **blueprints** or **schematics** that describe how the building blocks of a problem fit together. We will call such programs as **schemas** or, alternatively, **representations** of problems.

Our goal in this session will be to relate these schemas or problem representations to much vaster state spaces than our previous session, and thus to consider how to learn to search effectively in these spaces (or whether even the idea of "search" still makes sense). To do so, we will visualize problem-solving as an adaptive process of navigating the surface of something called a **performance landscape**, in which solutions map to different "elevations" that correspond to higher or lower performance.

Here too, we will see two paradigmatically-different approaches. In a first approach closer to the reinforcement paradigm — which we will call the **genetic approach** for its inspiration from how genes and chromosomes evolve — adaptation is viewed as simply

another type of search process. In a second approach closer to the learning-in-a-society paradigm — which we will call the **biological systems approach** for its inspiration in how evolution happens at the level of more complex units such as cells or organisms — adaptation is viewed as requiring distinct ways of manipulating symbols beyond search.

2. Adaptation: linking evolution to problem-solving in AI systems and organizations

Evolutionary strategies in AI systems and organizations can be traced to the work of Charles Darwin in the 19th century. Though Darwinism is familiarly thought of in terms of animals and plants, it is a general approach to thinking about adaptation in artificial systems as well. Darwinism implies that a fundamental way of thinking about the intelligence of any complex system is its "**fitness**", meaning its ability to **survive** and **reproduce** relative to other systems. Fitness depends on the **ability to adapt** to changes in the environment. In the parlance of our course, adaptation is essentially a generate-and-test strategy for assigning credit at the *population-level* (e.g., among a population of pin factories or organisms in a species).

2.1. The magic of adaptation

In AI, the magic of evolution is that some simple rules such as the market mechanism — and more generally called **evolutionary operators** — can be astonishingly powerful for adapting solutions to problems of vast complexity. These operators are commonly summarized as **VSR**, standing for *variation* (generating different solutions), *selection* (assigning credit to high-fitness (i.e., high-performing) solutions), and *retention* (using high-performing solutions in the next "generation" of trials).

Consider that what Babbage would have observed in early 19th century England was a vast number of competing factories. Whether by design or not, each of these factories would divide labor a bit differently (variation). Their differences would, in turn, lead to differences in the quality and cost of their pins. Credit would be assigned by market forces (selection). Factories that made higher quality or cheaper pins would thrive (retention). Factories that made lower quality or more expensive pins would go bankrupt. Over time, the market would "learn" to assign credit (i.e., select) high-performing pin factories.

In evolutionary terms, we would say that the firms that thrive have higher "fitness". In the AI field today, performance is often evaluated using benchmarks claimed to correspond to some **absolute measure of intelligence**. Yet consider our mouse and the maze. Its "intelligence" in learning to find the cheese was *relative* to how its actions "fit" the mazes that were given to it. Even if its learning were to generalize to all mazes, the mouse would still be specialized for maze problems rather than all problems in the

universe. For this course, we will likewise think of performance in AI and organizations similarly in terms of "fitness" of solutions to problems *relative to the specific type of problem*. The intuition is similar to how, in evolution, "survival of the fittest" applies to organisms that perform well (i.e., survive and reproduce) in specific niches (e.g., hippos in the tropics; penguins in Antarctica) rather than the whole world.

2.2 Visualizing adaptive processes with performance landscapes

The process of adaptively search for higher fitness in a problem-solving environment can be usefully visualized through the metaphor of a performance landscape. A **performance landscape** is a picture of the relationship between an agent's actions (e.g., choices or observations) and its performance with respect to a goal. The **Y-axis** (the "elevation" of the landscape) is the **measure of performance**. Higher "elevation" corresponds to higher performance. Though there may be multiple goals, we will assume that they are aggregated into a single performance measure captured by the Y-axis. The **X-axis** (the bottom of the landscape) is a set of **actions**, each of which has one or more **states** (i.e., possible values). As an example, imagine a laptop design problem that involves four components, the screen, the CPU, the memory, and the keyboard, each of which have 10 possible types. The actions would be choices or observations of the four components, and the states would be whichever of the 10 types are chosen or observed.

If there is only one action on the X-axis, the "landscape" is a 2D depiction of its relationship to performance. If there are two dimensions on the X-axis, then the landscape becomes 3D, and so forth. More typically, solving real-world problems involves taking vastly more than two actions. Since humans cannot easily visualize high dimensionality, however, a landscape is essentially a highly simplified visualization of the dimensions of the X-axis to 2D or 3D.

Hence, performance landscapes are used in research on AI and organizations as visually intuitive ways of depicting the feedback-driven processes (e.g., trial-and-error) by which an agent learns to take actions to achieve goals — *not as precise depictions*. They allow visualizing the learning process metaphorically as a "journey" across the surface of a landscape. The "journey" begins when the agent takes some initial actions. The particular actions that the agent takes lead to lower or higher performance, corresponding to points at various elevations on the landscape. The agent modifies its actions based on feedback about its performance (i.e., about its current position on the landscape), "journeying" to different points on the landscape.

To construct a landscape, we first need to define a performance landscape with respect to a goal related to some problem to be solved. The problem may be specific (e.g., how to get from A to B) or general (e.g., how to navigate in general). The "real" landscape is

called an agent's **task environment**. For any real-world problem, there will be a vast number of possible actions and performance criteria that belong to the agent's task environment. Given limited processing capacity, the agent considers only a small number of actions and performance criteria at any one time. The agent hence works with only **simplified, imperfect symbolic representations** of its task environment. We will refer to these simplified representations of symbols as the agent's **schemas**, though they are sometimes referred to as models, candidate solutions, programs, plans, schematics, frameworks or other terms.

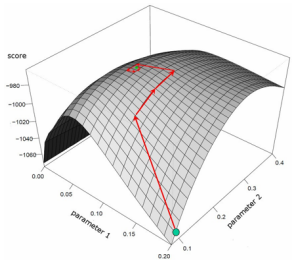
To interpret a landscape, we need to describe (1) a performance goal; (2) the *set of actions and states* in the schema; (3) *procedures for selecting actions* in response to feedback; (4) *procedures for learning to modify the procedures (e.g., setting a learning rate)*. Different AI ideas correspond to different procedures. In reinforcement learning, the action process is based on a policy, such as the mouse's policy of when to turn left or right in the T-maze. In search, the basic process is the use of heuristics. We will see other ideas throughout the course. Most generally, action processes are based on encoding and decoding bit strings in Shannon information, and interpreting lists in Simon information.

3. Landscape topologies

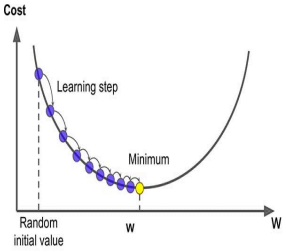
The topology of a landscape depicts differences in the quality of solutions in a population that we may expect to see. The "landscape" can be composed of "hills", "valleys", or "mesas" or any other features one can imagine. We can depict this as a "single peaked" landscape, with gentle slopes, a landscape with multiple fitness peaks, called a "rugged" landscape, or one with no peaks and just thresholds (a "mesa" landscape).

3.1 Single-peaked landscapes: hill-climbing (a.k.a., gradient descent)

Perhaps the simplest "journey" in a performance landscape is a simple trial-and-error learning strategy. The trial-and-error feedback can be depicted by "**hills**" or "**gradients**" in the landscape that depict a basis for trial-and-error learning. The intuition is that hills or gradients refer to points in the landscape that are close together, yet where there is a noticeable performance difference. An agent whose current actions map to a position on a "hill" in the landscape can "learn" to reach a nearby point just by moderately modifying actions. If the performance measure is some value to maximize, then this process is called "**hill-climbing**". If the performance measure is something to minimize, then the landscape is simply viewed upside-down, and the learning process is called "**gradient descent**".



A 3D view of hill-climbing



A 2D view of gradient descent

In this simplest landscape, there is just one "**global peak**" of performance, also called the **global optimum** (or, alternatively, a single gradient to descend — the **global minimum**). In a single-peaked landscape, the agent smoothly "hill-climbs" to the peak of the landscape — trial-and-error will always lead to the global optimum.

Though simple to depict, it does not mean that learning in such a landscape is necessarily easy. Remember that a 2D or 3D depiction of the landscape is just a visual aid, whereas actual landscapes are hyperdimensional. If the number of interdependent actions is large, then hill-climbing to the peak or gradient descent to the minimum may involve a steady yet incredibly long journey. For example, in organizations, the single peak may refer to the design of some complex product or system of activities that only becomes competitive in the market if some single optimal configuration is achieved.

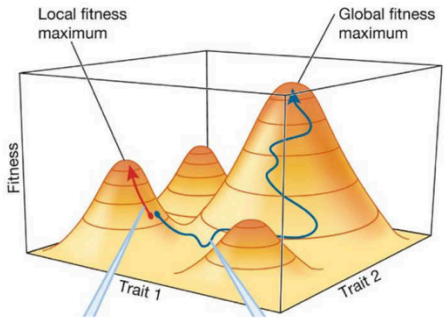
3.2 Rugged landscapes: hill-climbing and long-jumps

Let us go back to the earlier example of designing four components of a laptop — the screen, CPU, memory, and keyboard. Let us call each component's effect on performance as its **contribution** to the "fitness" of the laptop (e.g., how valuable the laptop would be for potential customers). For each laptop component, however, what we really need to do is specify a more complex **contribution function** to account for how the contributions are **interdependent** with other components. For example, the contribution of the CPU component to the value of the laptop depends in part on whether it is fast enough to support the memory component. Similar to how the multiple moves made by the mouse in the labyrinth were interdependent, the possible configurations increase combinatorially. It will be difficult to evaluate (i.e., assign credit to) all of them.

Due to interdependence and combinatorial complexity, even small changes to just a single component can have large effects on the performance (i.e., fitness) of the laptop. Simulations show that performance differences — gradients or hills — arise in similar types of configurations, even if none of them are optimal. These clusters of performance are called "**local peaks**" in the landscape, or simply "**local maxima**", and make the landscape look "**rugged**". Given that the laptop designer initially starts somewhere

randomly on the landscape, if there is a local peak nearby, it will be tempting to just "hill-climb" to it. Once at the local peak, there is no more gradient, and the learning stops. For example, a laptop designer might choose a screen size that is not ideal for customers, but then simply modify the other components (CPU, memory, keyboard) to "fit" the screen size. Such a laptop would be optimal *given* the choice of screen size, but not "globally" optimal. This is called being "**stuck**" or "**frozen**" on a local peak.

Such learning is called "**myopic**", in that the agent is only seeing things locally, rather than the entire landscape, and hence missing the global optimum. Let us further assume that actions are modified in the "neighborhood" of previous actions (e.g., as assumed in hill-climbing), then agents just climb to the nearest peak. Hence, the costs of myopia depend on the luck of wherever our initial actions map to in the performance landscape. This is called **path dependence**, meaning that early actions have a large effect on the overall learning process. Finally, you should see that a rugged landscape also helps visualize the phenomenon of **superstitious learning**. When we go up the "wrong" local peak, it is because we are getting feedback that our performance is improving, yet this feedback may be just superstition or at best partially true.



A rugged landscape showing the possibility of getting "stuck" on a local peak.

Clearly, to avoid myopia and path dependence, we need to be able to move off of local peaks to other peaks until, hopefully, we discover the global maximum. As distinct from incremental hill-climbing within a peak, we will call such movement in the landscape as "**long-jumping**". To do so requires making bigger modifications to actions than in hill-climbing — modifications that are not just informed by immediate feedback, but are more risky (hence, the "jumping" metaphor). We will call such actions as "**exploration**", in that we are venturing into a part of the landscape about which we do not have good feedback. Conversely, we will call smaller modifications — hill-climbing actions — as "**exploitation**", in the sense that such actions exploit the feedback immediately available to us. Though exploitation clearly puts us at risk for myopia and path dependence, exploration carries its own risks of venturing into the relative unknown. For example, the

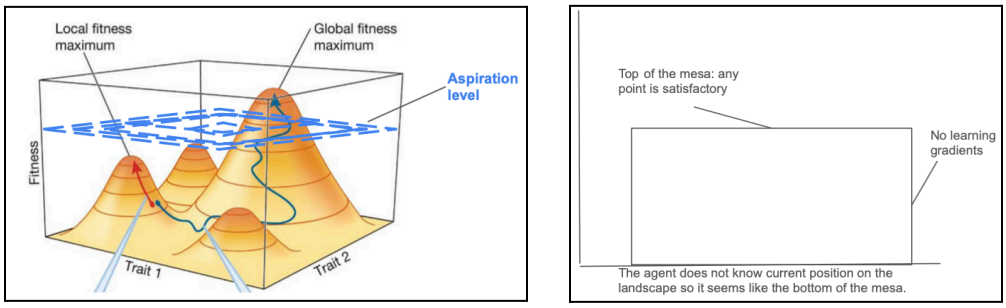
laptop designer could make a bold new change to the screen component, but may not have any reliable feedback on whether that change will increase or decrease fitness. The presence of multiple "local peaks" of performance in a "rugged" landscape thus raises a fundamental tradeoff for learning in AI and organizations.

Given that both "exploring" and "exploiting" have risks, the idea in learning programs in AI and in organizations is often to *balance* the two strategically. The ideal balance depends on many factors, such as the rate or degree of change in the environment, the skills of the agent to "explore" intelligently, or the importance of finding an optimal "combination". We classically see this tradeoff in organizations, for example, in the areas of **product innovation** and **strategy**, where managers continually have to decide whether to invest their budgets in doing what they do a bit better, or risk doing something different.

3.3 Satisficing and the landscape: lowering expectations vs. taking giant strides

We can depict satisficing in a performance landscape simply as a threshold for performance, reflecting Simon's term of an **aspiration level**. Any point higher than the threshold is satisfactory (and, hence, stops the adaptive "journey" through the landscape), and any point below the threshold is not satisfactory (such that the "journey" continues). Let us think of satisficing, however, in two different ways.

In one way, we can think of satisficing as simply *lowering our expectations* for desired fitness for a given problem. Here, the adaptive actions can be exploratory or exploitative, just as in rugged landscapes. You can think of this aspiration level as being higher than some local peaks, but lower than the global peak. Hence, the benefit of satisficing is that, even while economizing on information processing, the agent is able to avoid the worst peaks, which may be "good enough".



Satisficing in rugged (left) and mesa (right) landscapes.

In a more fundamental way, however, we can think of a satisfactory solution with respect to solving a problem that may be so complex that we may not even know quite how to start. For example, in the laptop design problem, the product development team may be

asked to build an entirely new kind of laptop for which it does not have closely related prior experience. Marvin Minsky referred to problem-solving environments of such complexity as having "mesa"-like landscapes. In a **mesa landscape**, the agent starts by taking actions that put it at some point on the bottom of the mesa (or so it seems to the agent — it cannot be sure if it is closer to the goal). The top of the landscape is basically flat, so the agent is not really concerned with the global optimum, or the risk of ending up on a local peak (the peak is, at most, a trivial journey once the flat top is reached, as depicted below). The agent somehow must get to some point at the flat top of the mesa — the satisfactory threshold.

Minsky referred to this threshold not so much as a personal aspiration level, but as some threshold satisfactory *understanding* of a problem required to even think about engaging in a search process. The key challenge here is different. Unlike a hilly or peaked landscape, there are no obvious slopes for hill-climbing or long-jumping to a satisfactory solution — *exploration-exploitation strategies will not work*. Minsky described the process required instead as one of taking "**giant strides**" in the landscape, in that one has to "climb the mesa" but generating and testing coherent pieces of evidence about what the problem is ("strides" signifying that actions are always grounded in some sort of coherent logical claim about the observations (i.e., data) for what to do next).

Characteristics of different landscape topologies

Landscape topology	Landscape features	Learning challenges	Learning strategies
Optimizing in single-peaked landscapes	One global peak, otherwise "smooth"	If interdependencies are large, vast info. processing required	**Hill-climbing / gradient descent**
Optimizing in rugged landscapes	Multiple local peaks	Myopia, path dependence, superstitious learning	Balance exploration (**long-jumping**) and Exploitation (**hill-climbing**)
Satisficing in rugged landscapes	Aspiration below global peak but above some local peaks	Setting a realistic aspiration level	Use heuristics to **set an aspiration level**
Satisficing in mesa landscapes	A flat bottom and a flat top, with no hills or gradients in between	Problems arise only as symptoms, where there are a lack of gradients for trial-and-error learning (sparse learning)	Taking "**giant strides**" by logically and coherently piecing together the problem.

4. Approaches to adapting on the landscape

What does the landscape metaphor imply for how to assign credit effectively? You can think of the approach to answering this question as driven by: *how well the agents of an AI system or organization understand, or need to understand, the interdependencies in the landscape*. We will explore two paradigmatically-different approaches, using three examples. The first two examples are closer to the reinforcement learning paradigm and, similarly, view learning to assign credit as fundamentally a matter of balancing the exploration and exploitation of actions. The third example is closer to the learning in a society paradigm, and views learning to assign credit as instead a matter of…

4.1 Approach 1a: gene-inspired representations

As a baseline approach, imagine a new category of watch emerging that has many interdependent features. Assume that the landscape is "rugged": only a small subset of possible configurations of features will be successful in the market. Customers' reactions are hard to predict. How should firms competing in this emerging category learn to assign credit to configurations they try out?

Our earlier example of competition among factories in Babbage's 19th century England implied "just let the market forces figure it out". Yet such an implication is not much comfort to a manager of one of the firms trying to innovate in this new watch category. It would mean that success or failure is essentially just a matter of randomly generating a variation of the watch, and then waiting to see what happens. The landscape metaphor implies, however, that a manager can to some extent "bring the market inside" the firm, based on using much the same logic as in reinforcement learning of balancing the exploration and exploitation of different watch configurations.

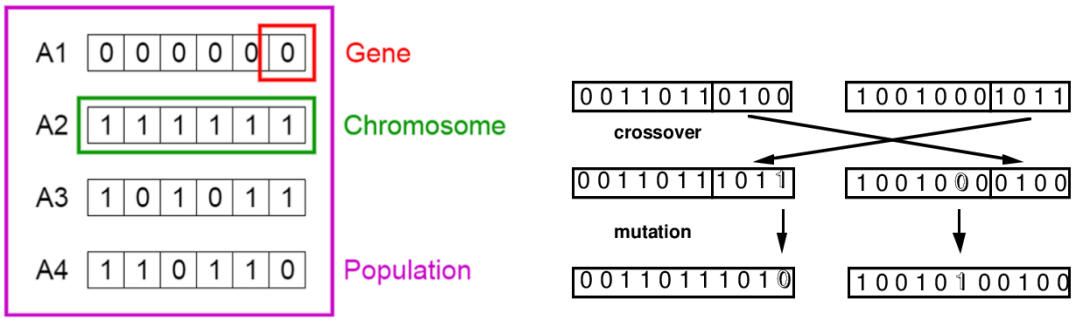
In such an approach, the firm might instead design an "**artificial selection environment**", where one treats the adaptive building blocks as **genes** associated with **individual traits** (i.e., akin to the choice of a product feature). This is much like the idea of plant-breeding as introduced by **Gregor Mendel**, in which evolution occurs through crossing-over (and occasionally mutating) genetic sequences across generations. In this view, the designer of an AI system or organization is basically a designer of **experiments**, and the strategy is how to vary radically different (exploratory) experiments from incrementally different (exploitative) ones.

This baseline approach assumes that a designer of an AI system or organization, while knowing that there is some pile of gene-like building blocks, only knows how to experiment with them, and cannot really rely on using any understanding of their structure (i.e., how the interdependencies can be decomposed). This is *analogous to*

Simon's second watchmaker. In contrast, if we assume that the firms *can* leverage some understanding of structure — like Simon's second, more intelligent watchmaker who leverages understanding of the watch components — then they can use abstractions to navigate the landscape more easily. In landscape terms, you can think of the effects of such use of components as "**smoothing**" the topology of the landscape from steep and rugged to gentler.

4.2 Approach 1b: chromosome-inspired representations

A first approach pioneered by the AI scholar John Holland at the University of Michigan is called "**genetic algorithms**". In genetic algorithms, a schema for representing a problem is analogized to a "chromosome", composed of sets of "genetic sequences". Chromosomes are coded in a computer simulation as bit strings. Each bit is analogized to a "gene", and each value of a bit is analogized to an "allele" (an expression of a gene). In a schema, the individual genes may be fixed as a one or zero, or allowed to vary across trials, in which case it is marked as an asterisk, "*". The asterisks represent the choice variables. For example, in a chromosome string, there could be six "genes", coded as 1****1. In this example, the first and last gene are fixed as "on" (having a value of one), and the four "genes" in the middle are asterisks, meaning they can vary over time. In the figure below, a population of four chromosomes are depicted, each with six genes, and where each gene is "on" (one) or "off" (zero).



Variation in the population of genes is coded by making random combinations of two or more chromosomes (called "**crossover**") and by randomly changing the values of one or more genes in a chromosome (called "**mutation**"). The fitness of the population is generated by coding for "natural selection" using statistical rules. In organizations, a gene can be thought of as encoding a "trait", such as a program for a product feature or a function. All traits are treated as decisions (implementation is assumed). The full set of genes (in a product, or in a strategy) is the "genome". A particular genome is called a "genotype", and can be analogized to a product design, an organization design, or a firm's strategy. Examples of crossover and mutation are depicted in the figure above on the right.

Genetic algorithms (GAs) are methods for transitioning from an initial population of *chromosomes* to (hopefully) a "fit" chromosome. The search process is adaptive in the sense that the "genes" are varied across a population, and then selected based on their fitness. This variation and selection can easily be carried out over many "generations" with a computer simulation, which depicts the distribution of performance. The "search space" is the set of candidate solutions. For example, for a set of 30 product features, the search space would be all combinations: an enormous amount.

Holland's key insight was that, just by the simple evolutionary mechanisms of crossover and mutation, a vast space of possible combinations of genes can be searched relative to what is possible in a conventional search. He called his insight "**implicit parallelism**", meaning that the "genetic" operators of crossover and mutation allowed considering most possible "chromosomes" in a population without having to try most of them. Further, he showed how genetic algorithms, or other schemes inspired by them, enable such effects simply based on balancing the level of **exploration-exploitation**. Hence, the idea is to use this same basic strategy as in reinforcement learning or the baseline "Mendelian" approach above, but just mix it with some basic abstractions (i.e., chromosomes instead of individual genes), *analogous to Simon's second watchmaker*.

4.3 Approach 2: biological systems-inspired schemas

A second approach, pioneered by Marvin Minsky's student **Gerald Sussman** at MIT, is called the "**propagator model**". In the propagator model, a schema for representing a problem is analogized to a biological system composed of a variety of cells. In an AI system or organization, cells are objects that can be coded using any symbols, not just bit strings. Symbols inside each object contain a variety of information, analogized to complex information involved in cellular processes.

Rather than information corresponding to 0,1 bits that correspond to genetic "traits" being turned on or off, the complex information about each object describes whether it satisfies or violates certain constraints. For example, in something like Simon's mechanical watch, an object could represent a watch component such as the watch display and encode various constraints, such as how fast the hour hand should turn or how tightly the spring should wind. Rather than a discrete set of genetic traits", these objects are connected, akin to how cells in an organism send signals to one another. For example, if the hour hand moves too fast, this information can be used to consider that the spring may be too tight.

Here, problem-solving is not about "searching" for good variations in traits through crossover and mutation, but about generating objects and then sending ("**propagating**") information about them to related objects to evaluate whether their local constraints are

satisfied. Hence, variation is not *across* an entire schema, but within *constraints* in the schema. As an evolutionary strategy, the key advantage is that adaptation can take place independently *within* a constraint, rather than having to evolve some entire schema. To go back to the factory analogy, the imagery here is not of some randomly varied population of factories where credit is assigned by market forces, but rather where credit can be assigned effectively within a factory based on trying out various ways of satisfying each stable constraint on the underlying problem.

The logic for this "propagator" model is captured by Minsky's overall view of evolutionary strategies as it relates to the performance landscape metaphor. For Minsky, the limits of both gene-inspired (i.e., Mendelian experimenting) or chromosome-inspired (i.e., genetic algorithms based on crossover and mutation) is that they are *far too slow* — they mostly "hill-climb", whereas what is needed for intelligence is some way to, closer to Simon's idea of searching based means-ends analysis, flexibly manipulate symbols to construct the landscape is.

Evolution, Minsky proposed, may lead to certain intelligent behaviors — e.g., animals that survive in real-world environments, or factories that can produce pins efficiently — but it is not by itself intelligent in the sense that we should want to think about since the processes take a vast number of generations. For most of the problems of intelligence that we face, in contrast, using standard evolutionary approaches will either get us *stuck on local peaks* of a landscape, or we will simply have no idea of what a peak even means. Hence, something like Sussman's propagator model is needed to construct an understanding of the structure of a problem in terms of stable functions (constraints) and how a variety of solutions might satisfy these constraints.